

FULL STACK DEVELOPMENT

FASE 4 | AULA 04 -

JEST

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS	13

EMANDA

O QUE VEM POR AÍ?

Nesta aula, vamos explorar o uso do Jest, uma das ferramentas mais populares para testes automatizados em JavaScript, especialmente em aplicações React Native. Entenderemos a importância dos testes automatizados e como configurar e utilizar o Jest para garantir a qualidade e a robustez do código.

No hands on, implementaremos testes unitários e de integração utilizando Jest, cobrindo desde a configuração inicial até a escrita de testes eficientes e significativos.

EXEMPLO

HANDS ON

Nesta aula, configuraremos o Jest em uma aplicação React Native para implementar testes automatizados. Inicialmente, instalaremos o Jest e suas dependências, configurando o ambiente de testes no projeto. Criaremos testes unitários para componentes individuais, validando a renderização correta e o comportamento esperado.

Também implementaremos testes de integração para garantir que diferentes partes do aplicativo funcionem juntas como esperado. Utilizaremos mocks para simular dependências externas e garantir que os testes sejam isolados e precisos.

Vamos explorar as funcionalidades do Jest, como snapshots, para verificar mudanças inesperadas na interface do usuário. Discutiremos as melhores práticas para escrever testes eficientes, incluindo a importância de uma boa cobertura de código e a execução regular dos testes durante o desenvolvimento e, por fim, concluiremos testando a aplicação para garantir que todos os testes passem e que o código esteja livre de bugs.

SAIBA MAIS

O Jest é uma estrutura de testes em JavaScript desenvolvida pelo Facebook, amplamente utilizada para testar aplicações React e React Native. Ele oferece um conjunto abrangente de ferramentas para escrever, executar e estruturar testes, incluindo testes unitários, de integração e de snapshot. Ele é conhecido por sua facilidade de uso, rápida execução de testes e integração robusta com o ecossistema React.

Por que usar Jest?

O Jest é projetado para ser fácil de configurar e usar, proporcionando uma experiência de testes integrada com React Native. Ele vem com uma configuração padrão que funciona bem para a maioria dos projetos, permitindo que desenvolvedores(as) comecem a escrever testes rapidamente sem a necessidade de configurações complexas.

O Jest também fornece relatórios de cobertura de código, que são essenciais para garantir que todas as partes críticas da aplicação sejam testadas. Esses relatórios mostram quais partes do código foram executadas durante os testes, ajudando a identificar áreas não testadas e melhorando a qualidade geral do software.

Além disso, uma das funcionalidades mais poderosas do Jest é a criação de testes de snapshot, que capturam a saída renderizada de componentes e a comparam com uma versão armazenada anteriormente. Isso é extremamente útil para detectar mudanças inesperadas na interface do usuário. Sempre que um componente é alterado, o teste de snapshot falha, indicando que a saída renderizada mudou.

Outro motivo para utilizá-lo se dá devido sua execução rápida e eficiente. Ele utiliza um recurso chamado "watch mode" que executa apenas os testes afetados pelas mudanças recentes no código. Além disso, o Jest é capaz de rodar testes em paralelo, aproveitando ao máximo o hardware disponível para reduzir o tempo de execução.

Quando usar Jest?

O Jest é ideal para testar qualquer aplicação JavaScript, mas especialmente eficaz em:

- Aplicações React e React Native: o Jest é perfeito para verificar componentes e fluxos de usuário, garantindo que a interface e a lógica da aplicação funcionem como esperado.
- APIs e serviços: é excelente para testes de integração e unitários para endpoints e lógica de negócios, garantindo que os serviços backend estejam respondendo corretamente.
- Bibliotecas e pacotes: ajuda a garantir que módulos individuais funcionem como esperado, facilitando a manutenção de bibliotecas e pacotes JavaScript.

A pirâmide de testes no contexto do Jest

A pirâmide de testes é um modelo conceitual que sugere uma abordagem equilibrada para a implementação de testes automatizados em um projeto de software. Ela é dividida em três níveis principais: testes unitários, testes de integração e testes de ponta a ponta (end-to-end). O Jest, uma poderosa ferramenta de teste para JavaScript, suporta efetivamente cada nível da pirâmide, permitindo que as pessoas desenvolvedoras possam criar uma suíte de testes abrangente e eficiente.

Testes unitários com Jest

Os testes unitários formam a **base da pirâmide** e são os mais numerosos. Eles se concentram em testar pequenas unidades de código, como funções, métodos ou componentes individuais, de maneira isolada. O Jest é particularmente eficaz para testes unitários devido à sua facilidade de uso, desempenho rápido e capacidade de isolar partes do código.

Vejamos algumas vantagens dos testes unitários com Jest:

Rapidez e eficiência: testes unitários são rápidos de escrever e executar. O Jest otimiza a execução desses testes, permitindo execuções frequentes durante o desenvolvimento.

Isolamento e precisão: com Jest, é fácil isolar funções ou componentes, utilizando mocks e stubs para simular dependências, garantindo que cada unidade seja testada de forma independente.

Cobertura de código: o Jest fornece relatórios detalhados de cobertura de código, ajudando a identificar quais partes do código foram testadas e quais precisam de mais atenção.

Testes de integração com Jest

Os testes de integração ocupam o **meio da pirâmide** e verificam como diferentes unidades de código funcionam juntas. Eles são menos numerosos que os testes unitários, mas mais complexos, pois envolvem a interação entre múltiplos componentes ou módulos. Algumas vantagens são:

Verificação de colaboração: com Jest, quem desenvolve pode testar a interação entre componentes ou módulos, garantindo que eles colaborem corretamente.

Deteção de problemas de interface: esses testes ajudam a identificar problemas que surgem quando componentes interagem, algo que os testes unitários podem não detectar.

Utilização de ferramentas adicionais: o Jest se integra bem com bibliotecas como `@testing-library/react` e `@testing-library/react-native`, facilitando a criação de testes de integração para aplicações React e React Native.

Testes de ponta a ponta (End-to-End) com Jest

Os testes de ponta a ponta estão no **topo da pirâmide** e são os menos numerosos devido à sua complexidade e custo de execução. Eles simulam o comportamento do usuário final, testando o sistema completo em um ambiente que replica o uso real. Algumas vantagens desses testes são:

Verificação de fluxos completo: esses testes garantem que todo o sistema funcione conforme esperado, desde a interface do usuário até a integração com o backend.

Simulação realista: utilizando frameworks como Detox junto com Jest, é possível simular interações do usuário em aplicações React Native, testando fluxos completos de ponta a ponta.

Confiança no produto final: embora sejam mais caros e demorados, esses testes fornecem um alto grau de confiança de que o sistema funcionará corretamente para o usuário final.

Como implementar - Configuração inicial do projeto

Criar projeto React Native:

1. Configure um novo projeto React Native utilizando o CLI do React Native.
2. Certifique-se de que todas as dependências estão instaladas e o ambiente de desenvolvimento está configurado.

Instalação do Jest:

1. Instale o Jest e suas dependências.

```
npm install --save-dev jest @testing-library/react-native @testing-library/jest-native
```

Configuração do Jest:

1. Configure o Jest no seu projeto adicionando um script de teste no package.json.

```
"scripts": {  
  "test": "jest"  
},  
"jest": {  
  "preset": "react-native",  
  "setupFilesAfterEnv": [  
    "@testing-library/jest-native/extend-expect"  
  ]  
}
```

Escrevendo testes unitários

Testes de componentes: escreva testes para componentes individuais, verificando se são renderizados corretamente e se comportam conforme o esperado.

Testes de integração

Testes de fluxos de usuário: implemente testes que verificam a interação entre diferentes componentes e partes do aplicativo, garantindo que funcionem bem juntos.

Mocks e snapshots

Utilizando mocks: use mocks para simular dependências externas e garantir que os testes sejam isolados.

Criando snapshots: crie e utilize snapshots para capturar a saída renderizada dos componentes e detectar mudanças inesperadas.

Estrutura do projeto:

```
|-- src/  
| |-- components/  
| | |-- MyComponent.js  
| | |-- MyComponent.test.js  
| |-- screens/  
| | |-- HomeScreen.js  
| |-- App.js  
|-- package.json
```

MyComponent:

```
import { View, Text, StyleSheet } from 'react-native';  
  
const MyComponent = ({ title }) => {  
  return (  
    <View style={styles.container}>  
      <Text style={styles.title}>{title}</Text>  
    </View>  
  );  
};  
  
const styles = StyleSheet.create({  
  container: {  
    padding: 20,  
    alignItems: 'center',
```

```
    },  
    title: {  
      fontSize: 18,  
    },  
  }  
});  
  
export default MyComponent;
```

Testes do componente MyComponent:

```
import { render } from '@testing-library/react-native';  
import MyComponent from './MyComponent';  
  
test('renders correctly with given title', () => {  
  const { getByText } = render(<MyComponent title="Test Title" />);  
  expect(getByText('Test Title')).toBeTruthy();  
});  
  
test('matches snapshot', () => {  
  const tree = render(<MyComponent title="Snapshot Title" />).toJSON();  
  expect(tree).toMatchSnapshot();  
});
```

HomeScreen:

```
import { View, StyleSheet } from 'react-native';  
import MyComponent from '../components/MyComponent';  
  
const HomeScreen = () => {  
  return (  
    <View style={styles.container}>  
      <MyComponent title="Home Screen" />  
    </View>  
  );  
};  
  
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
    justifyContent: 'center',  
    alignItems: 'center',
```

```
    },  
  });  
  
export default HomeScreen;
```

App.js:

```
import { SafeAreaView, StyleSheet } from 'react-native';  
import HomeScreen from './screens/HomeScreen';  
  
const App = () => {  
  return (  
    <SafeAreaView style={styles.container}>  
      <HomeScreen />  
    </SafeAreaView>  
  );  
};  
  
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
  },  
});  
  
export default App;
```

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, aprendemos sobre a importância e a implementação de testes automatizados utilizando Jest em aplicações React Native. Discutimos como o Jest facilita a escrita e a execução de testes unitários e de integração, garantindo a qualidade e a robustez do código. Implementamos testes para componentes individuais e fluxos de usuário, utilizando mocks para simular dependências e snapshots para verificar mudanças na interface do usuário.

Abordamos as melhores práticas para escrever testes eficientes, incluindo a importância de uma boa cobertura de código e a execução regular dos testes durante o desenvolvimento. Concluimos que o Jest é uma ferramenta poderosa para garantir a estabilidade e a confiabilidade das aplicações React Native.

REFERÊNCIAS

JEST. **Getting started**. Disponível em: <https://archive.jestjs.io/docs/en/22.x/getting-started.html>. [s.d.]. Acesso em: 29 jul. 2024.

LINDER, A. **Mastering React Native**: Full Stack Mobile App Development with React Native. [s.l.]: Packt Publishing, 2020.

OHNS, E. **React Native for Mobile Development: Harness the power of React Native to build efficient and feature-rich mobile applications**. [s.l.]: Packt Publishing, 2020.

PALAVRAS-CHAVE

Palavras-chave: React native. Front-end. Jest.

EMENDAS

POSTECH